depends on that of another, make sure that you order the modifiers in the right sequence. In Ballotv3.sol, which uses the access modifiers `validPhase` and `onlyChair`, the `validPhase` modifier may take precedence and be applied first. In other words, if the phase is incorrect, you don't have to check who is invoking the `register()` function. The header of the `register()` function, therefore, becomes

```
Function register(address voter) public validPhase(Phase.Regs)
                                          onlyChair
```

Here's another example from an online buying use case:

```
function buy(..) payable enoughMoney itemAvail returns (..)
```

A function call to `buy()` verifies whether enough money is available (with the `enoughMoney` modifier) before checking the availability of the item. If enough money is not available, the function reverts without checking the item's availability through the `itemAvail` modifier.

3.5 Assert() declarations

So far, our discussion of modifiers has involved two special built-in functions of Solidity: `require()` and `revert()`. In this section, you'll learn about one more special function, `assert()`, which asserts that a condition is met during the computation process inside a function.

Suppose that you would like at least three votes (or majority votes) to be cast for the winning proposal in the ballot problem we've been discussing. You can enforce this rule by using an `assert()` clause at the end of the `reqWinner()` function. You